Linux Service and Process Supervisor

Team No: 22 | Section: [S5] |

Members: Sai kiran – 2520030125, Harshith – 2520030393, Nishanth – 2520030323

Guide: Dr. Harika

Operating Systems and Systems Programming (OSSP) | KLH Bachupally Campus, CSE | PBL 2026–27

Abstract

- This project addresses the issues of unmanaged daemon crashes, application deadlocks, and zombie process leaks tnd developers. It provides robust, low-level process supervision using C11 and POSIX primitives, including POCRIX primitives, fork(), execy(), exec(), setsid(), axynchronous signal-safe trapping (sigation, waitpid aemon, walted with WNOHANG), and event-driven I/O: rant(), and event-driven I/O multiplexing with select().

Problem Statement & Objectives

- Problem: Unmonitored background service crashes, application deadlocks, and zombie process leaks exhaust system resources and cause unmanaged server downtime.

Objective 1: Implement process session isolation and non-blocking zombie reaping using fork(), execvp(), setsid(), and waitpid(WNOHANG).

Objective 2: Build a single-threaded event loop with select() to multiplex CLI control and pipe-based heartbeat liveness monitoring.

Objective 3: Automate fault recovery using exponential backoff restarts while tracking CPU/memory usage via getrusage().

Scope: Linux administrators and systems developers; covers process lifecycles, POSIX signals, anonymous pipes, and resource accounting; strictly non-GUI and single-threaded (no pthreads).

Existing vs Proposed System

Aspect	Basic / Existing Approach	Proposed Project
Process handling	Manual or limited execution, unmonitored crash	Automated supervisor state machine (FSM), active tracking
Communication	No / isolated execution, no status telemetry	Anonymous pipes (FD 3) for periodic active heartbeats
Resource handling	High CPU polling, zombie process table leak	Multiplexed non-blocking I/O via select(), zombie reaping via waitpid()
Error handling	Undetected silent crash, infinite deadlock hang	Trap events (SIGCHLD), automatic timeouts, backoff recovery restarts
Key gap	No central watchdog, manually intensive crash/deadlock recovery	Zero-dependency self-healing event-driven daemon supervision

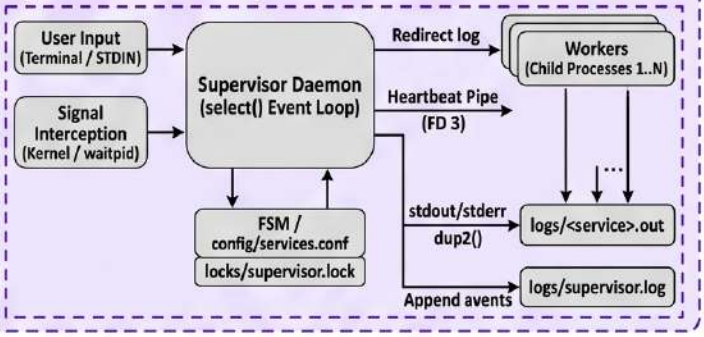
Feasibility Study

- **Technical:** Developed in ISO C11, GNU toolchain (GCC, Make) on Ubuntu/Linux (WSL) utilizing POSIX APIs.
- **Economic:** Open-source tools (zero software cost), minimal server hosting overhead (<5MB memory footprint).
- **Operational:** CLI prompt allows administrators status monitoring and process control (pause/resume); unbuffered logs (/logs) for debugging.
- **Schedule:** Phase 1 (reqs & parsing), Phase 2 (process & signal handling), Phase 3 (event loop & health watchdog), Phase 4 (testing & deployment).

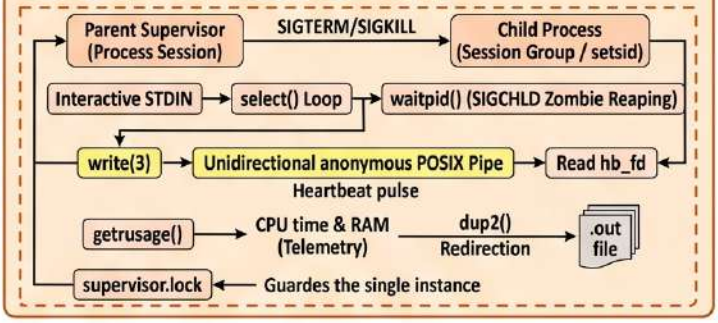
Tools & Technologies

- **Programming:** C (ISO C11), Bash (Shell) scripting, Makefile
- **Platform:** Linux / Ubuntu (native, WSL)
- **OS Concepts:** Processes & State Machines, IPC (Anonymous Pipes), Event-driven I/O Multiplexing, Signals, Virtual Memory, File-locking, Resource Telemetry
- **System Calls / APIs: fork(), execvp(), waitpid(), pipe(), setsid(), sigaction(), killpg(), select(), dup2(), open(), write(), close(), getrusage(), fcntl()**
- **Tools:** GCC, Make, GDB, Git & GitHub, VS Code

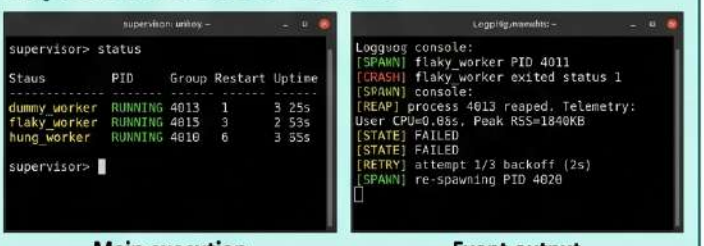
System Workflow / Architecture



Process, IPC & Resource Design



Implementation Screenshots



Main execution | Event output

Results & Testing

- Successfully validates all FSM state transitions (RUNNING, PAUSED, FAILED, UNHEALTHY, STOPPED) during simulation
- Tested crash recovery using flaky_worker: successfully trapped terminations, reaped zombies, and triggered backoff retry policy
- Boundary validation ensures corrupted configs, invalid args, or duplicate daemon launches (supervisor.lock) fail gracefully without supervisor crash.

Conclusion

- Fully implements high-availability zero-dependency service monitor daemon using C11 and POSIX APIs.
- Leverages OSSP concepts like async signal traps, pipes, FSM tracking, and waitpid() reaping to minimize human intervention.
- Guarantees daemon stays highly efficient, maintaining under 5MB RAM consumption for 24/7 self-healing watchdogs

Future Scope

- Incorporate system containerization/cgroups for strict CPU/memory resource boundaries.
- Develop dynamic configuration hot-reloading using SIGHUP without terminating stable worker tasks.
- Expand administration controls via Socket-based TCP/UNIX networks for remote command prompt dashboards

References: [1] Linux man-pages (man7.org)
[2] Stevens, W. Richard. "Advanced Programming in the UNIX Environment"
[3] GNU glibc and toolchain documentation

QR Code →